

Modern LLM architecture step-by-step



Karina

Research Engineer at Nebius

More than 3 years of experience with LLMs and NLP

<https://www.linkedin.com/in/karina-zainullina/>

BPE (Byte Pair Encoding)

The algorithm:

- Define the target vocabulary size **N** (the number of tokens we want)
- Initialize the list of tokens with the bytes
- Find the most frequent token pair in the training corpus, add it to the list of tokens
- Repeat until we get **N** tokens.

Example:

Corpus = ["low", "lower", "lowest"]; **N**=10

0. Start: **vocab** = [#l, #o, #w, #e, #r, #s, #t]

1. The most frequent:

lo = #l + #o and **ow** = #o + #w

vocab = [#l, #o, #w, #e, #r, #s, #t, #lo]

Note: now **low** = #lo + #w

2. The most frequent: **low** = #lo + #w

vocab = [#l, #o, #w, #e, #r, #s, #t, #lo, #low]

3. The most frequent: **lowe** = #low + #e

vocab = [#l, #o, #w, #e, #r, #s, #t, #lo, #low, #lowe]

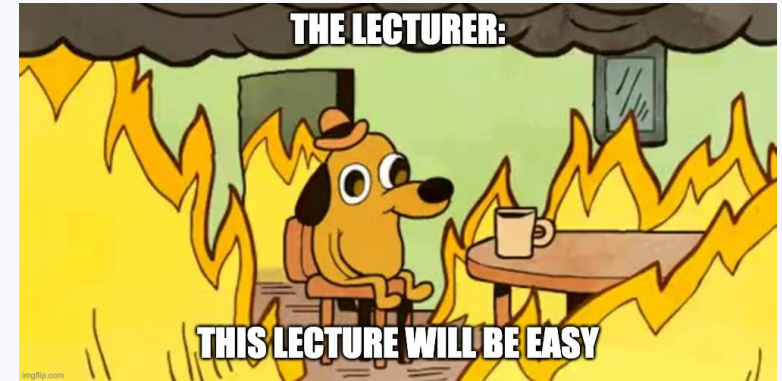
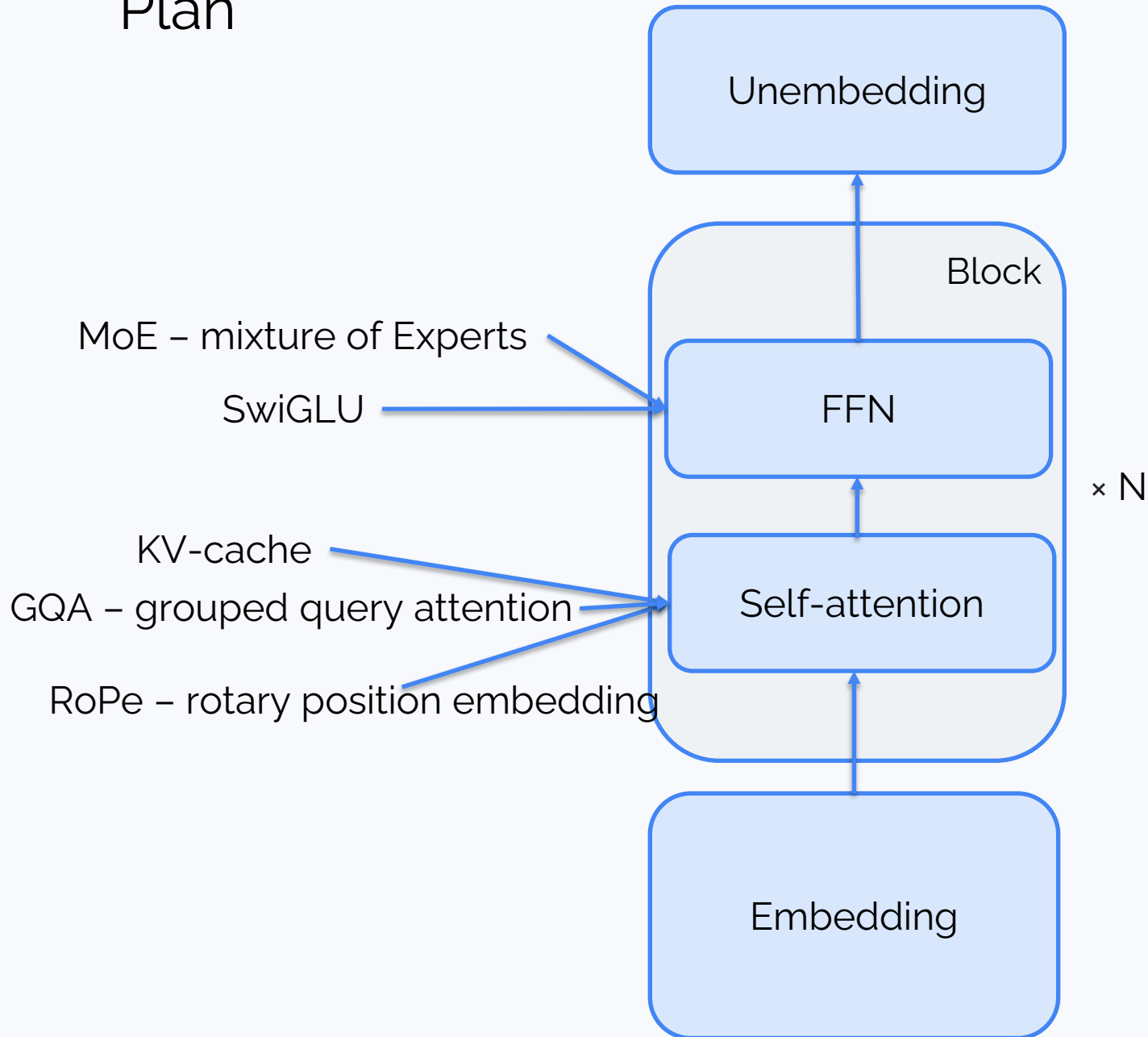
10 tokens now! Finally:

low = #low

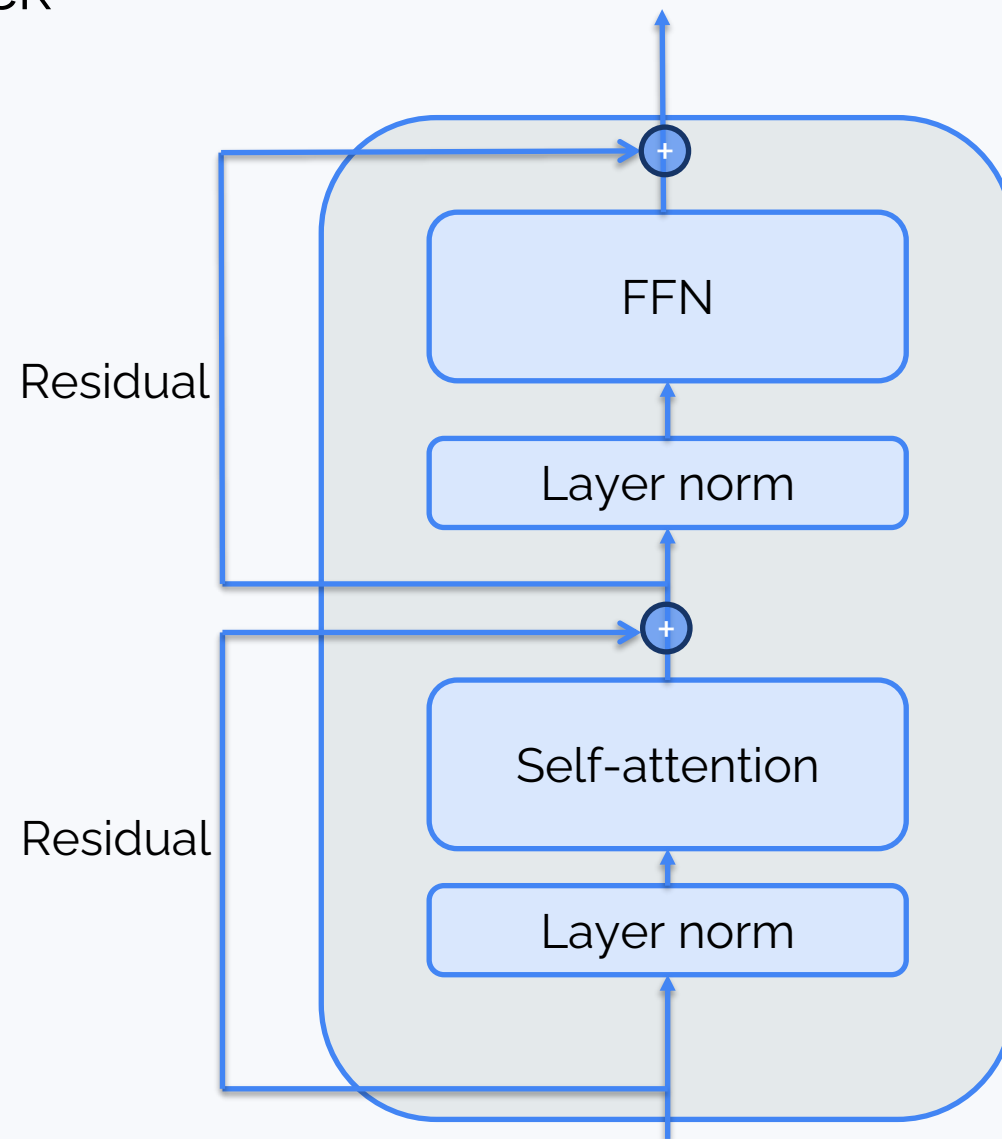
lower = #lowe #r

lowest = #lowe #s #t

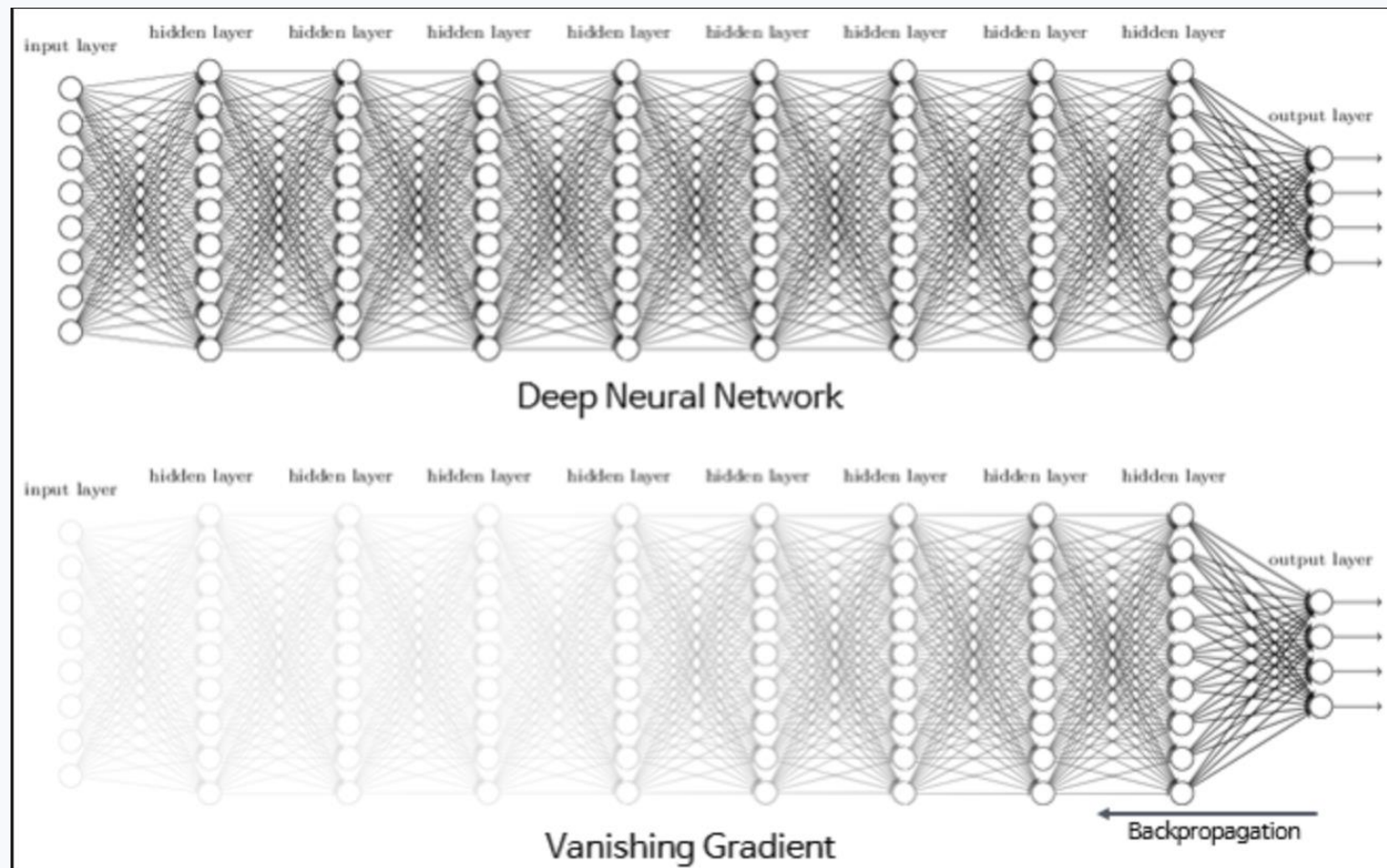
Plan



Modern LLM block



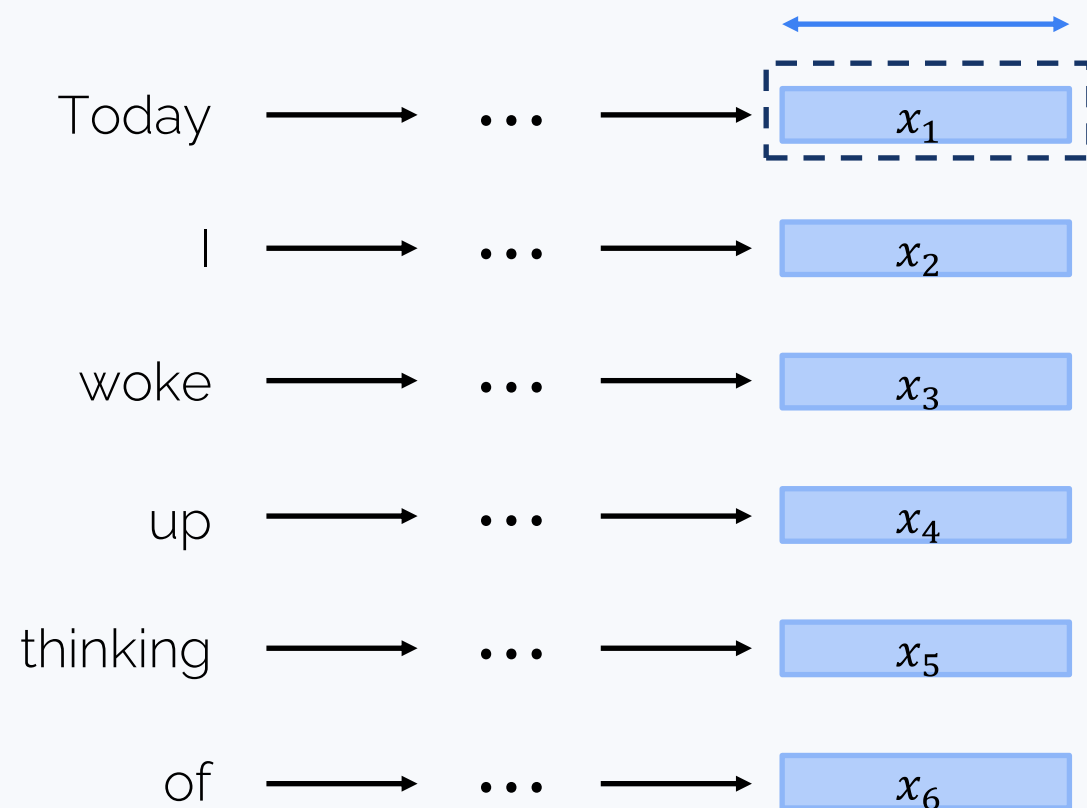
Residual, what for?



Layer normalization

- Normalize each vector, making
- the average of its coordinates = 0
 - its std = 1

Then apply trainable renormalization



Layers

$$y_1 = \frac{x_1 - \mathbb{E}[x_1]}{\sqrt{\text{Var}[x] + \epsilon}} \cdot \gamma + \beta$$

Now widely used RMSNorm:

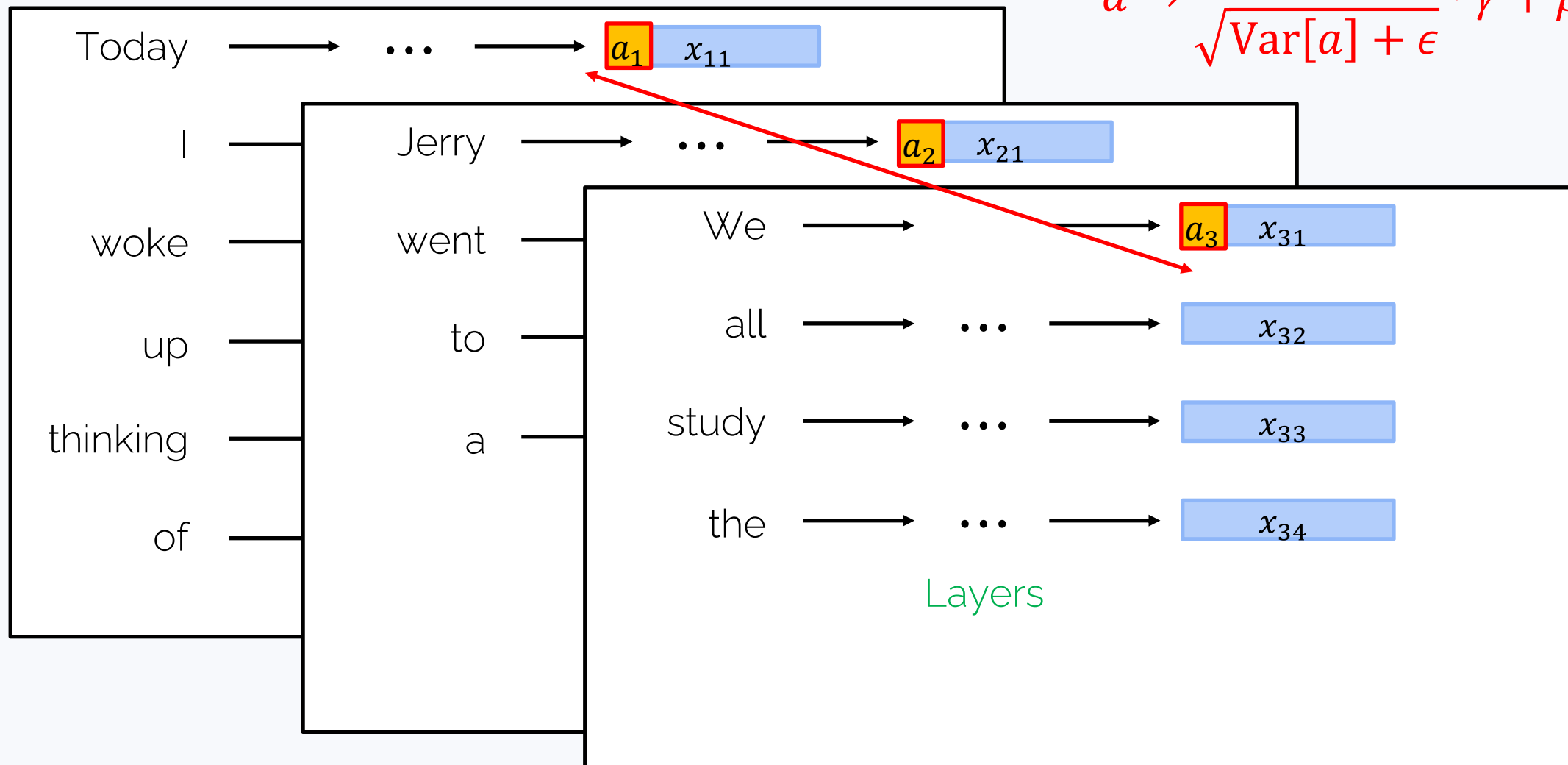
$$y = \frac{x}{\text{RMS}(\mathbf{x})} \gamma, \quad \text{where} \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}.$$

trainable

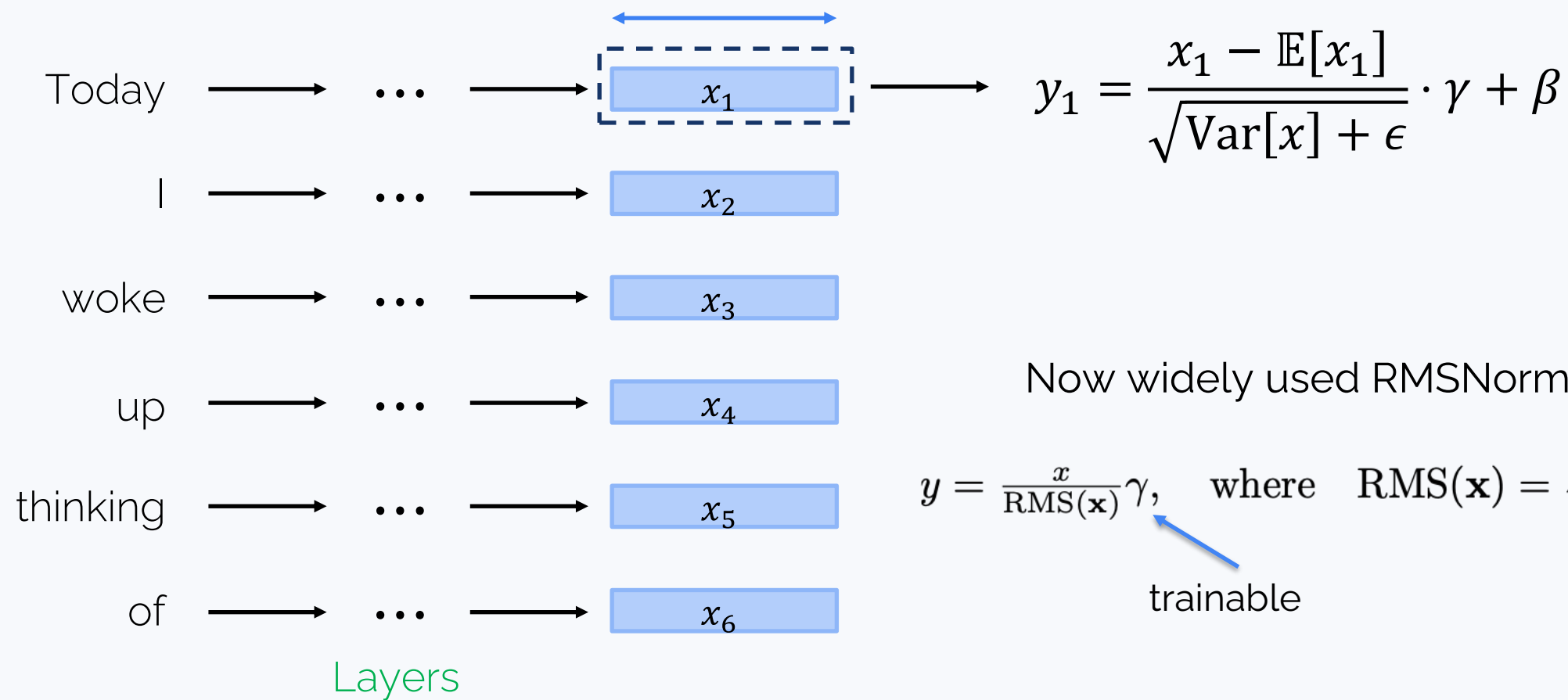
Batch normalization

Normalize each coordinate along the batch
For the first coordinate:

$$a \rightarrow \frac{a - \mathbb{E}[a]}{\sqrt{\text{Var}[a] + \epsilon}} \cdot \gamma + \beta$$



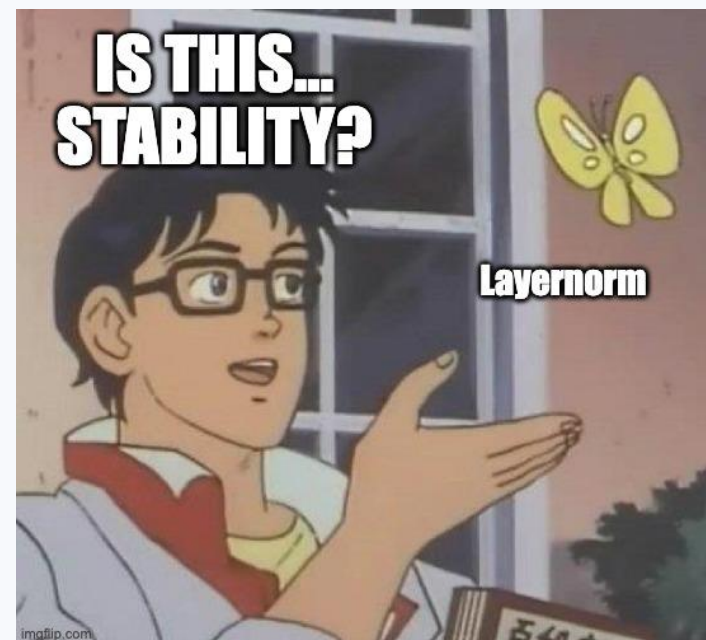
Layer normalization



Layer Norm, what for?

$$y = \frac{x}{\text{RMS}(\mathbf{x})} \gamma, \quad \text{where} \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}.$$

1. Stabilizes training
2. Prevents Exploding Gradients
3. Improves Model Generalization

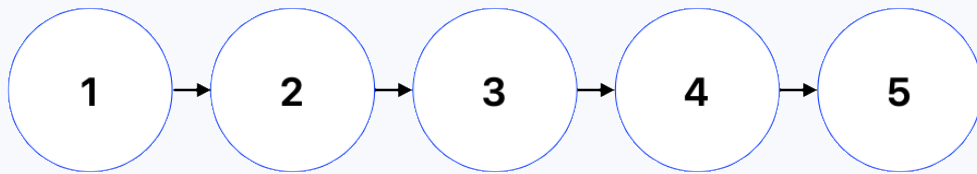


Positional embedding

Positional embedding

Absolute

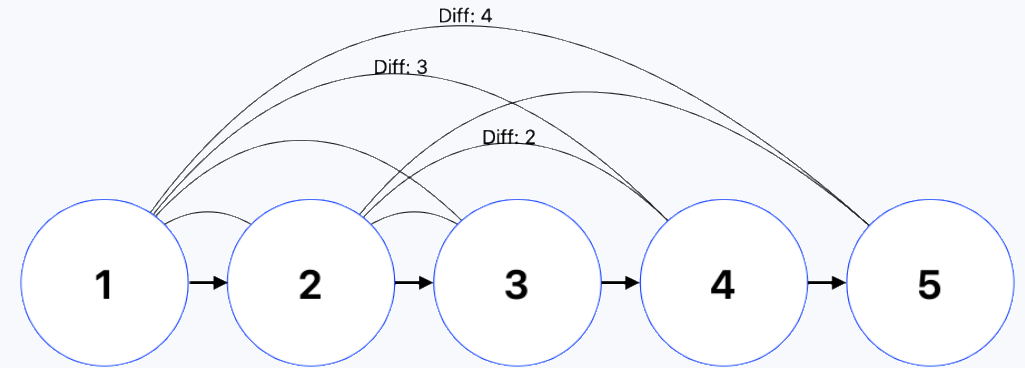
Encodes absolute positions of tokens using fixed embeddings added to the input.



$$f_{t:t \in \{q,k,v\}}(x_i, i) := (x_i + \boxed{p_i}) \mathbf{W}_{t:t \in \{q,k,v\}}$$

Relative

Encodes positional relationships between tokens, often applied in self-attention



$$\text{score}(i, j) = \frac{(q_i \cdot k_j^T + q_i \cdot \boxed{a_{ij}^T} + p_{ij})}{\sqrt{d}}$$

Rotary position embedding

Counterclockwise rotation by angle φ around the origin is a linear transformation of the 2d plane $\mathbb{R}^2$:

$$(x, y) \mapsto (x, y) \cdot \begin{pmatrix} \cos \varphi & \sin \varphi \\ -\sin \varphi & \cos \varphi \end{pmatrix} \quad (x, y) \cdot e^{i\varphi} \text{ is the same as } (x, y) \cdot \begin{pmatrix} \cos \varphi & \sin \varphi \\ -\sin \varphi & \cos \varphi \end{pmatrix}$$

So, lets assume

$$f_q(x_m, m) = x_m W_Q e^{im\theta} = Q_m e^{im\theta}, \quad f_k(x_n, n) = x_n W_K e^{in\theta} = K_n e^{in\theta}$$

$$f_q(x_m, m) \cdot f_k(x_n, n)^\top = Q_m \begin{pmatrix} \cos m\theta & \sin m\theta \\ -\sin m\theta & \cos m\theta \end{pmatrix} \cdot \begin{pmatrix} \cos n\theta & -\sin n\theta \\ \sin n\theta & \cos n\theta \end{pmatrix} K_n^\top =$$

$$Q_m \begin{pmatrix} \cos(m\theta - n\theta) & \sin(m\theta - n\theta) \\ -\sin(m\theta - n\theta) & \cos(m\theta - n\theta) \end{pmatrix} K_n^\top = Q_m e^{i(m-n)\theta} K_n^\top$$

Rotary position embedding

We use a block-diagonal rotation matrix:

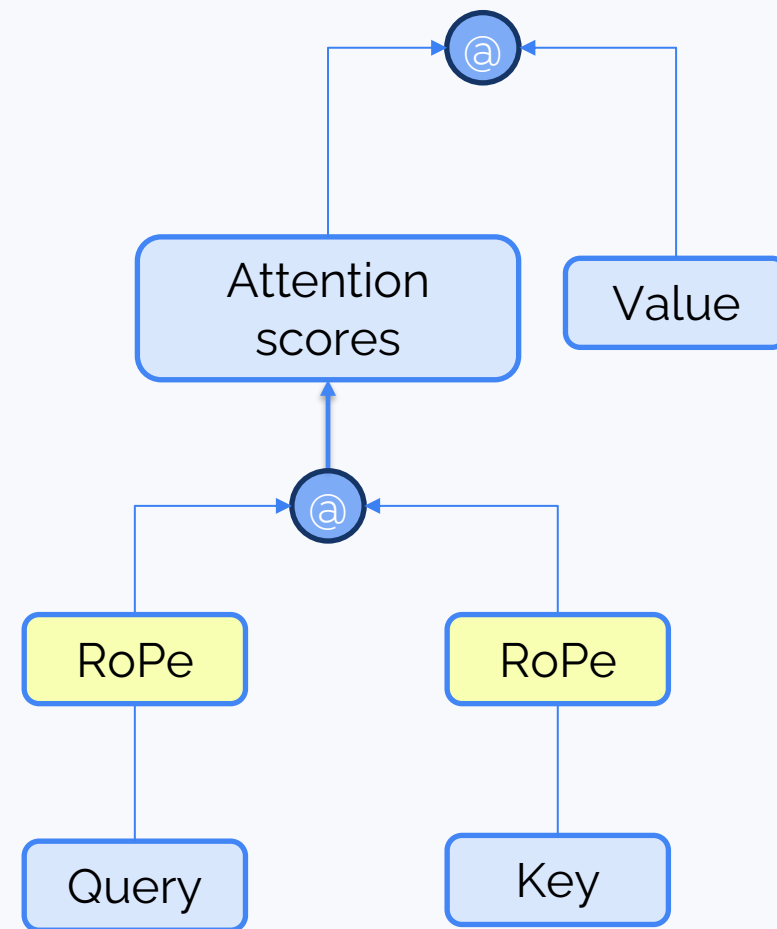
$$f_q(x_m, m) = x_m W_Q R_{\Theta, m}^d, \quad f_k(x_n, n) = x_n W_K R_{\Theta, n}^d,$$

where

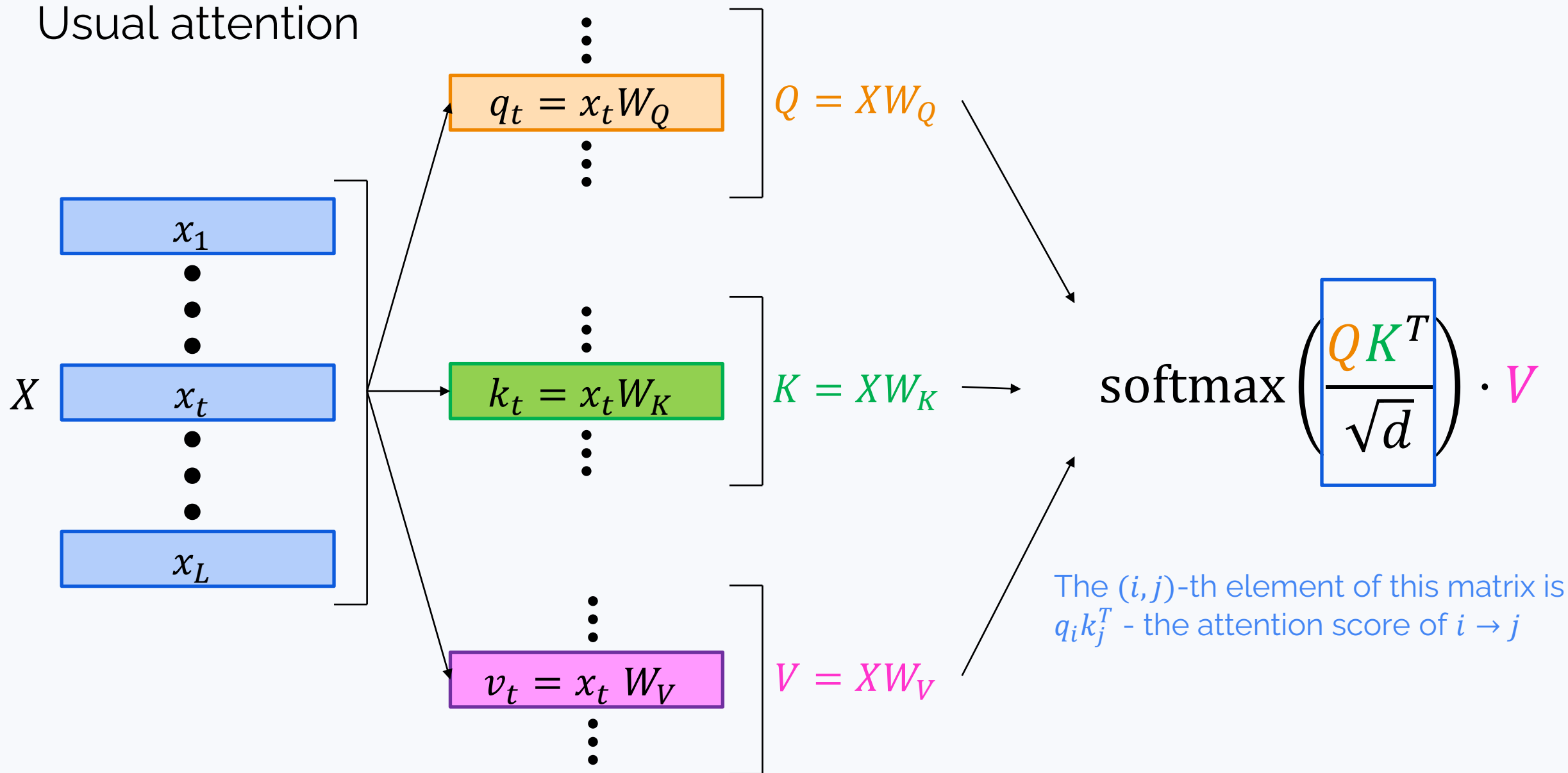
$$R_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & \sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ -\sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & \sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & -\sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} & \sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & -\sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix},$$

where the parameters Θ are set to

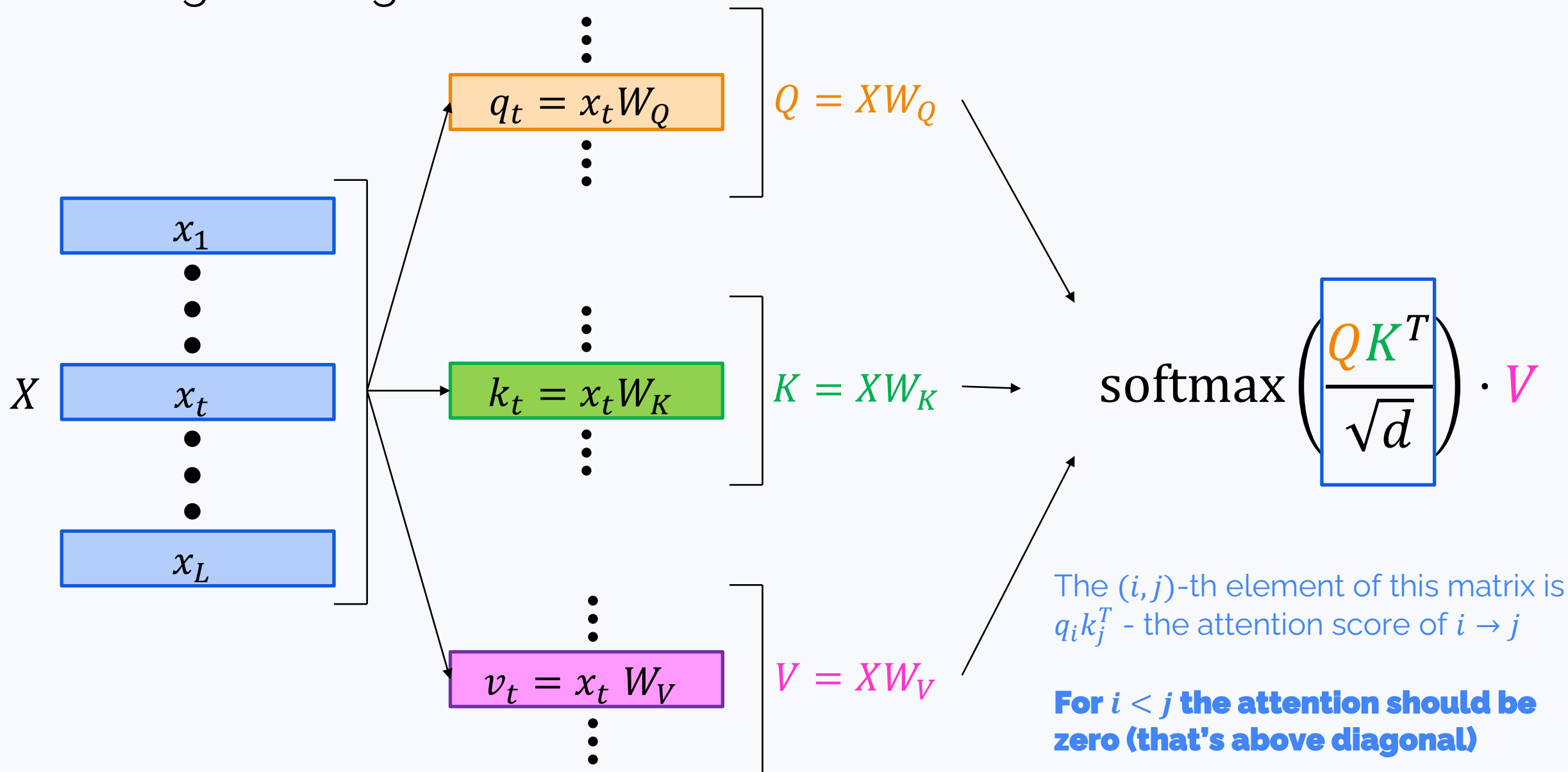
$$\theta_i = \theta_{\text{base}}^{-2(i-1)/d}.$$



Usual attention



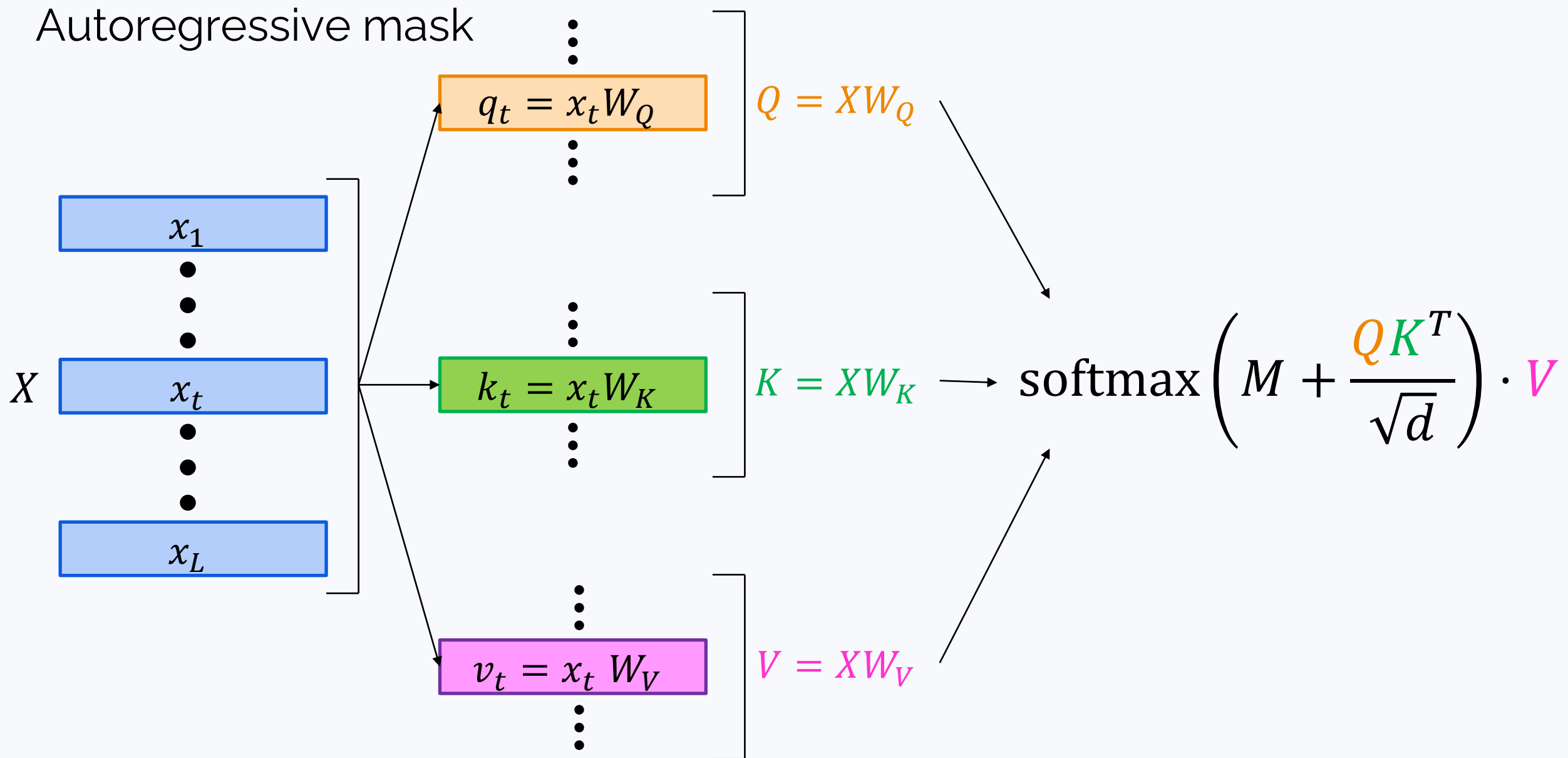
Autoregressive generation



Autoregressive mask

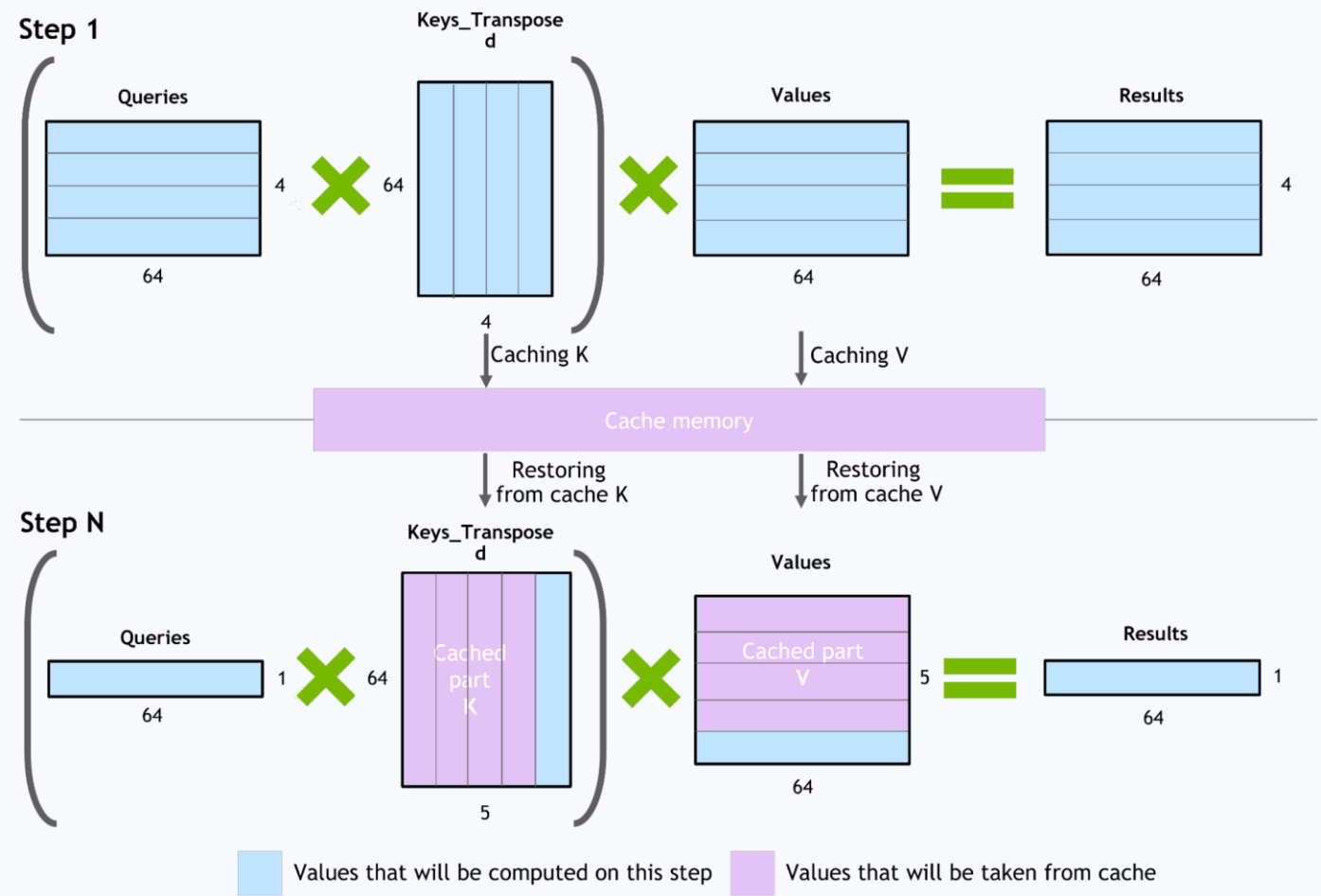
$$\text{softmax} \left(\begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 0 \\ \hline 1 & 1 & 0 & 0 \\ \hline 1 & 1 & 1 & 0 \\ \hline 1 & 1 & 1 & 1 \\ \hline \end{array} \otimes \left[\begin{array}{c} \boxed{q_1} \\ \vdots \\ \boxed{q_L} \\ \hline \boxed{k_1} \quad \dots \quad \boxed{k_L} \\ \sqrt{d} \end{array} \right] \right) \begin{array}{c} \boxed{v_1} \\ \vdots \\ \boxed{v_L} \end{array}$$

Autoregressive mask



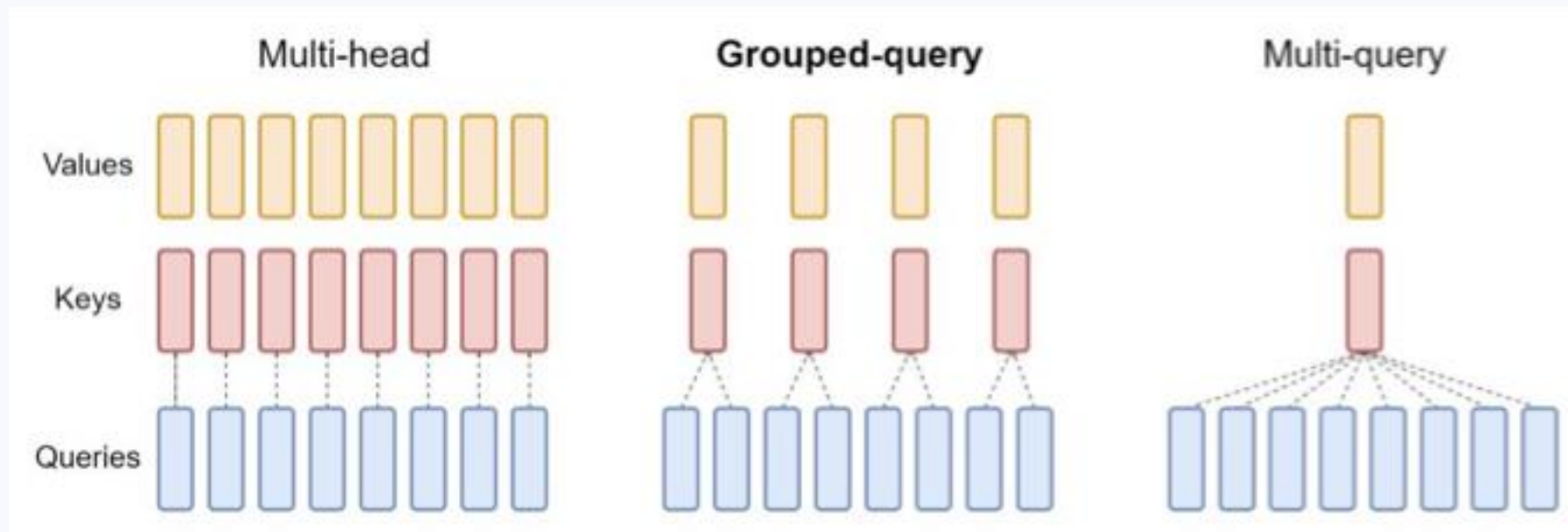
Key Value cache

$(Q * K^T) * V$ computation process with caching

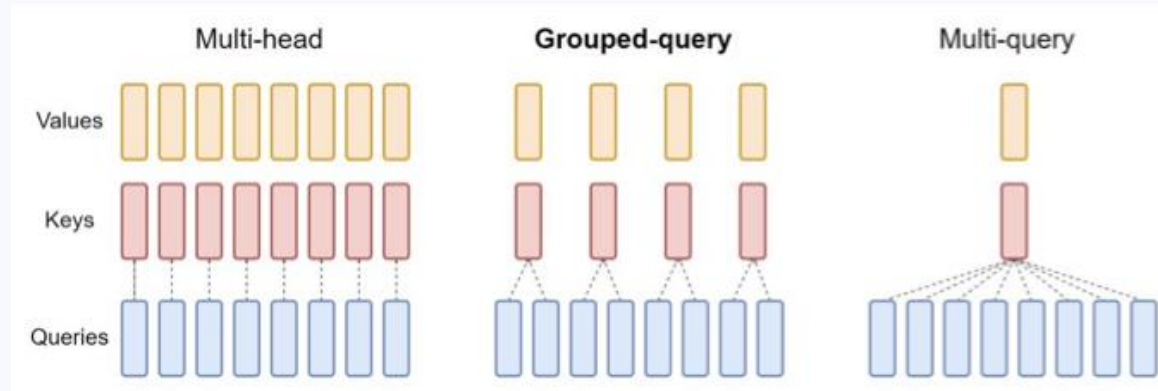


"hidden_size": 8192
"max_position_embeddings": 131072
"num_attention_heads": 64
"num_key_value_heads": 8
"num_hidden_layers": 80

Grouped Query Attention



Grouped Query Attention



What for?

- To reduce the size of the key-value cache and therefore, amount of data that needs to be loaded
- GQA is higher quality than MQA but faster than MHA
- Standard sharding for large models replicates the single key and value head by the number of model partitions. GQA removes the waste from such partitioning

FFN

$$\text{FFN}(x, W, W_2, b, b_2) = \text{ReLU}(xW + b)W_2 + b_2, \text{ where } \text{ReLU}(x) = \max(0, x)$$

not smooth at 0

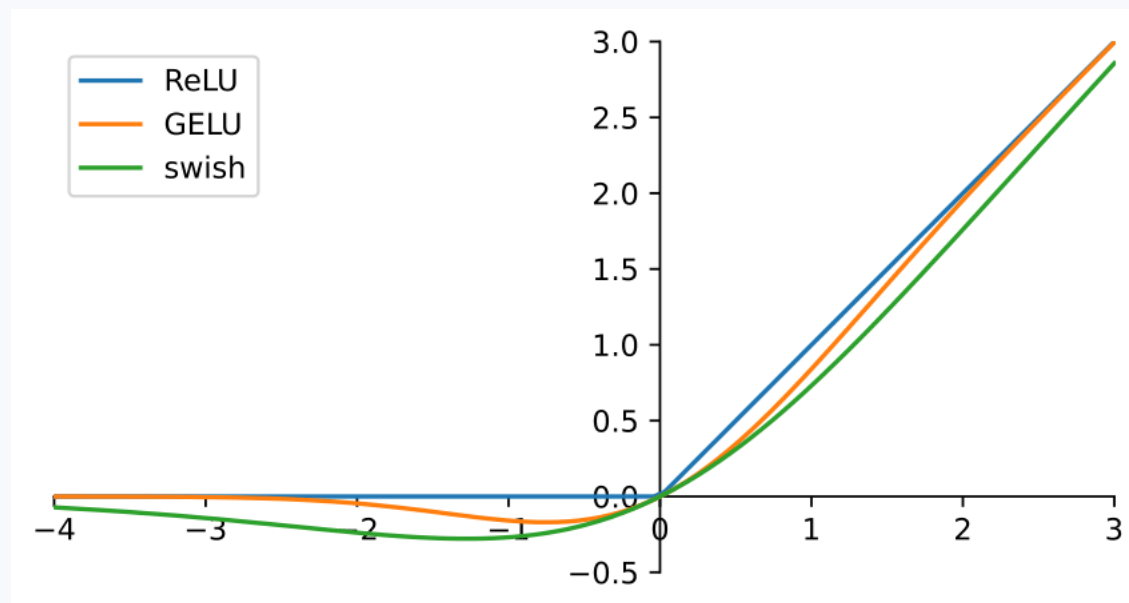


$$\text{GELU}(x) = x\mathbb{P}\{\xi \geq x\} = x\Phi(x), \text{ where } \xi \sim \mathcal{N}(0, 1) \text{ and } \Phi \text{ is the cdf of a standard Gaussian distribution.}$$

computationally expensive



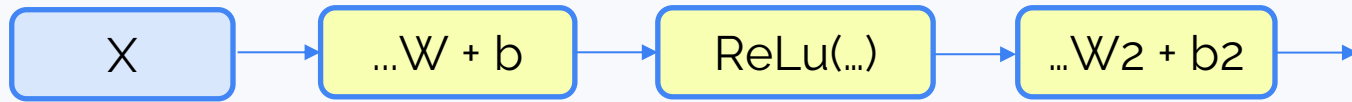
$$\text{Swish}_\beta(x) = x \cdot \sigma(\beta x)$$



FFN

MLP:

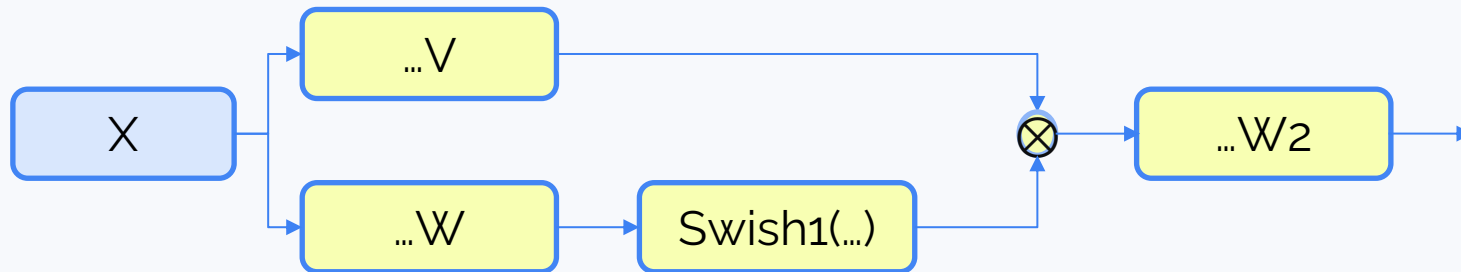
$$\text{FFN}(x, W, W_2, b, b_2) = \text{ReLU}(xW + b)W_2 + b_2, \text{ where } \text{ReLU}(x) = \max(0, x)$$



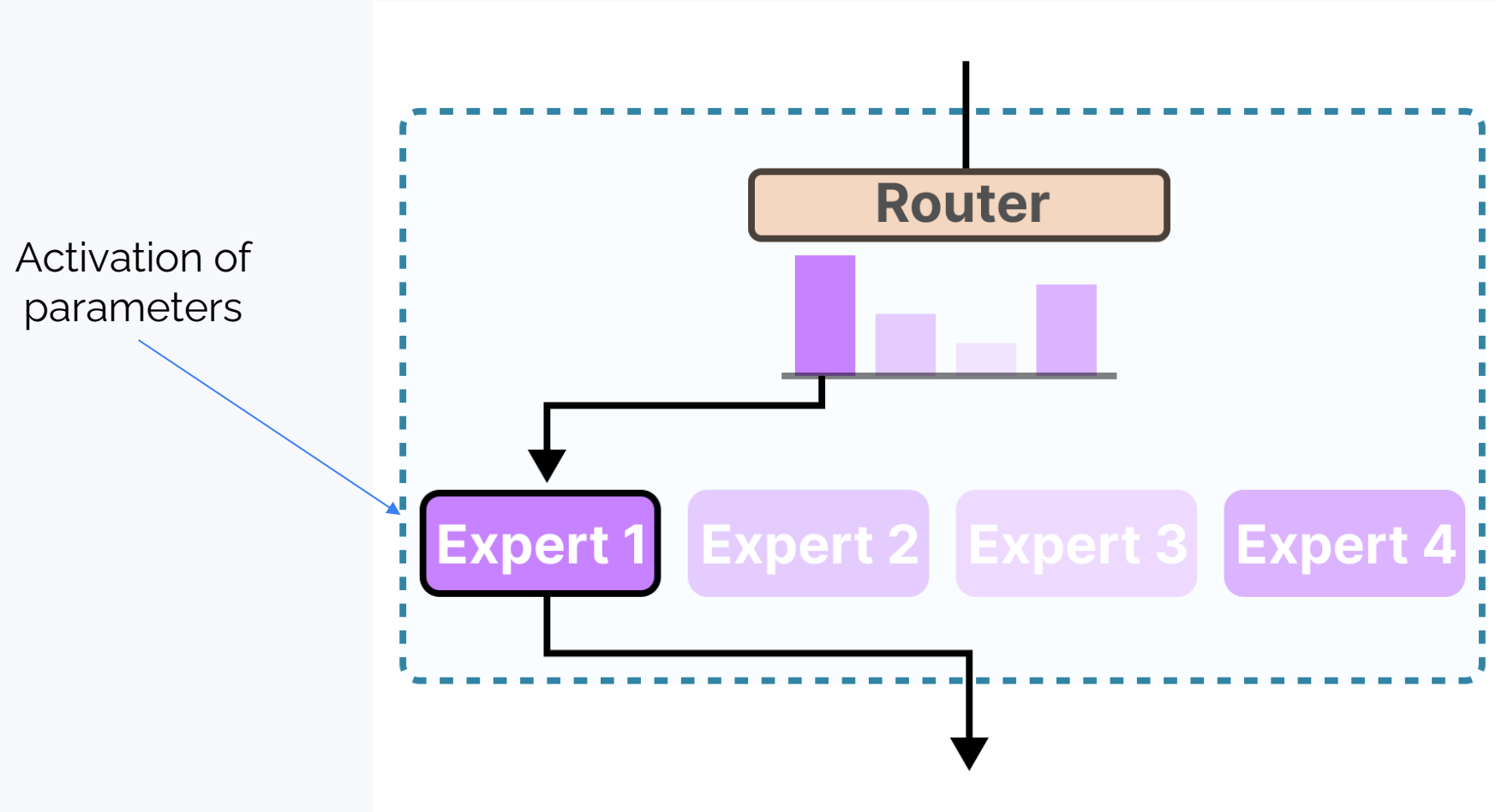
SwiGLU:

$$\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)W_2$$

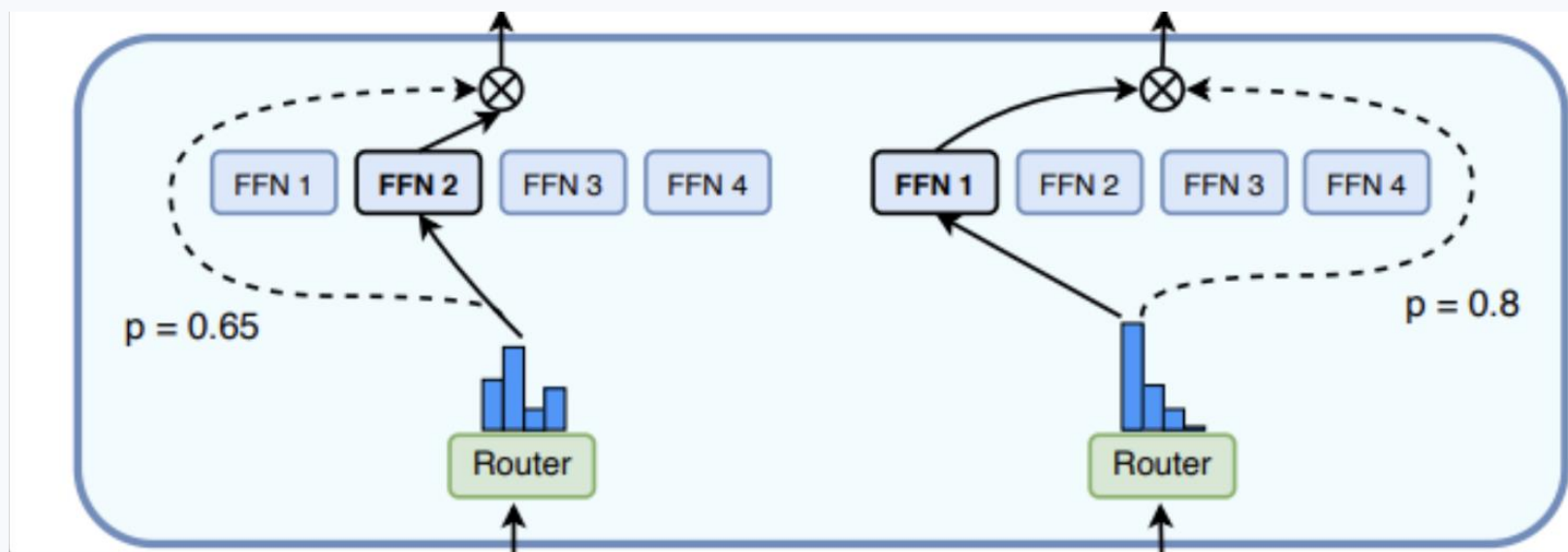
↖ elementwise product



Mixture of experts



Mixture of experts



Mixture of experts

N - number of experts, K - number of experts activated per 1 token

Input: x - inner representation of 1 token

Gating logits: $z(x) = \mathbf{w}^\top \mathbf{x} + b$, where $\mathbf{w}, b$ - trainable router

$g_i(x) = \text{softmax}(x) = \frac{\exp(z_i)}{\sum_{j=1}^N \exp(z_j)}$ - gating function

The final output of MoE block is:

$y = \sum_{i=1}^K g_i(x) \cdot y_i$, where $y_i = E_i(x)$ - output of expert i with input x

*In most cases experts have capacity (how many tokens can come to one expert). If the capacity is full, token is skipped by MoE block.

Mixture of experts

Why does it matter? Let's compare Mistral 7b and Mixtral 45B (8 x 7B)

- Mixtral requires ~ 6 times more memory than Mistral
- In Mixtral 14B (out of 45B) parameters are activated at each inference. In Mistral all 7B parameters are activated
- Mixtral few times more powerful than Mistral

Mixture of experts

Why does it matter? Let's compare Mistral 7b and Mixtral 45B (8 x 7B)

- Mixtral requires ~ 6 times more memory than Mistral
- In Mixtral 14B (out of 45B) parameters are activated at each inference. In Mistral all 7B parameters are activated
- Mixtral few times more powerful than Mistral



Mixture of experts

Problems:

Training Instabilities

- *Sparse Activation*: MoE often activates only a subset of experts for a given input, which can lead to unstable gradients and complicate training.
- *Balancing Experts*: The gating network may not distribute work evenly across experts, causing some to be over-utilized while others remain under-utilized.
- *Dead Experts*: Experts that are rarely activated may not get sufficient training, resulting in suboptimal performance.

Scalability Issues

- *Model Scaling*: While MoE can scale well with more experts, the communication overhead between components (e.g., experts and gating network) can become a bottleneck.

Mixture of experts

Balancing experts, dead experts:

- Auxiliary loss

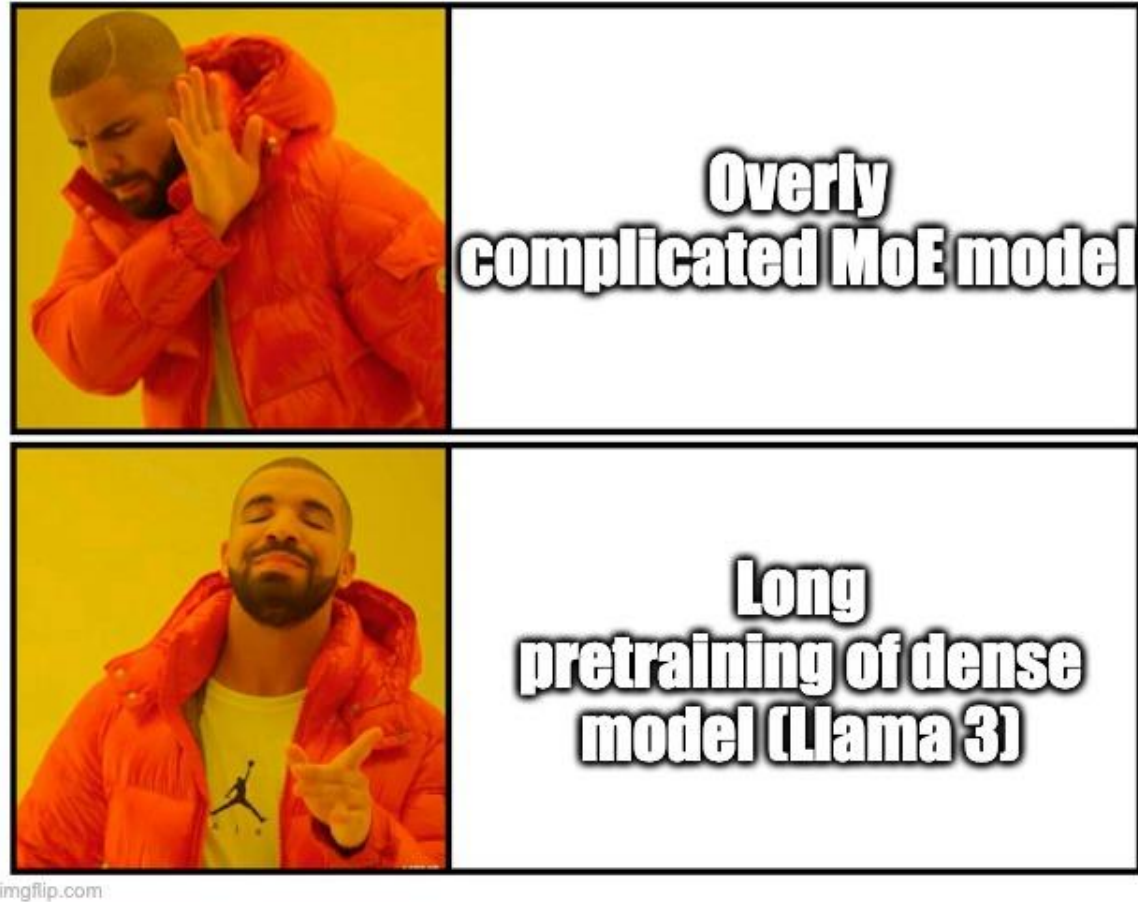
$$\mathcal{L}_{\text{Balance}} = \alpha \sum_{i=1}^N f_i P_i, \quad f_i = \frac{N}{KT} \sum_{t=1}^T \mathbf{1}(\text{Token } t \text{ selects Expert } i), \quad P_i = \frac{1}{T} \sum_{t=1}^T s_{i,t}$$

T - a sequence of length, N is the total number of experts, K is the number of experts selected for each token, $s_{i,t}$ is the routing score, f_i represents the fraction of tokens routed to Expert i and P_i denotes the average gating scores of Expert i

- Randomness for load balancing

$$H(x)_i := (x \cdot W_g)_i + \text{StandardNormal}() \cdot \text{SoftPlus}((x \cdot W_{\text{noise}})_i), \quad H(x)_i - \text{router logits}$$

Mixture of experts



Thank you